klhhuxsu8

April 2, 2026

1 FNN

[2]:
```
import os
os.environ['CUDA_VISIBLE_DEVICES']='0'
import torch
import torch.nn as nn
# import torch.nn.functional as F
import numpy as np
import pandas as pd

import time
# import scipy.io as io
from sklearn.metrics import mean_squared_error, mean_absolute_error

import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

plt.rcParams['font.sans-serif']=['Times New Roman'] #
plt.rcParams['axes.unicode_minus']=False

torch.manual_seed(1234)
np.random.seed(1234)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('running on: %s'%(device))
```

```
running on: cuda
```

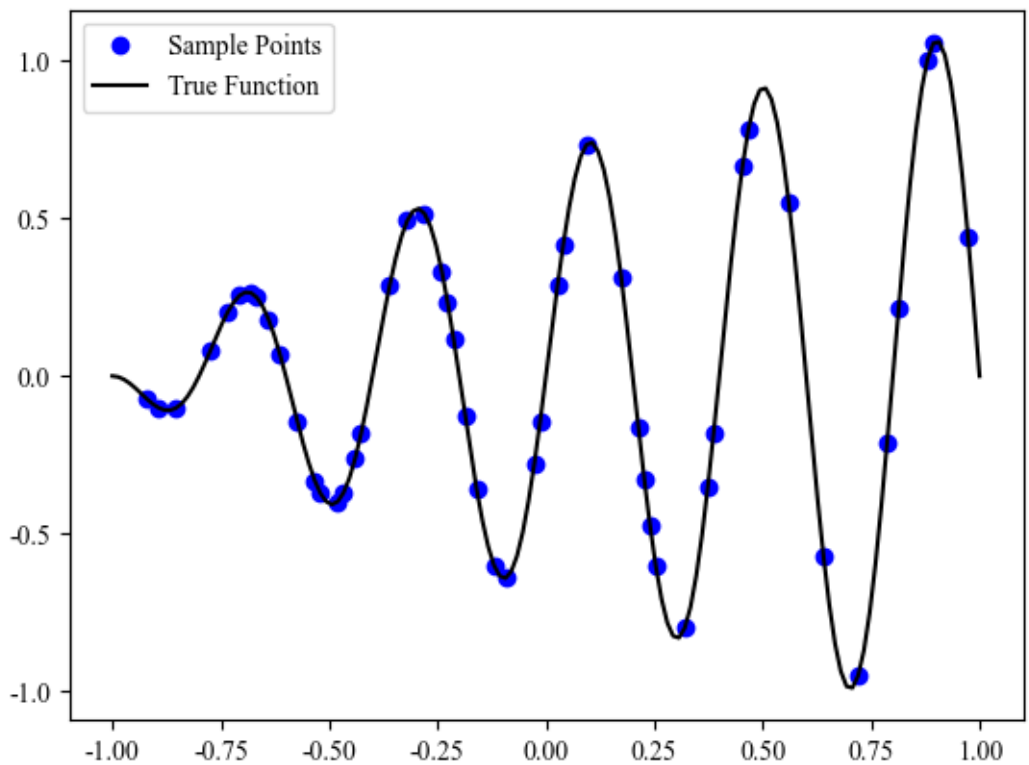
1.1 FNN

[3]:
```
## ---- FNN ----
class FNN(nn.Module):
    def __init__(self, layers, actn = 'tanh'):
        super().__init__()
        L = len(layers)
```

```
        self.linear = nn.ModuleList([nn.Linear(layers[l-1], layers[l]) for l in␣
↪range(1, L)])

        #
        if actn == 'relu':
            actn = nn.ReLU()
        elif actn == 'tanh':
            actn = nn.Tanh()
        elif actn == 'sigmoid':
            actn = nn.Sigmoid()
        self.activation = actn

    def forward(self, inputs):
        x = inputs
        for linear in self.linear[:-1]:
            x = self.activation(linear(x))
        x = self.linear[-1](x)
        return x
```

1.2

[4]:
```
#
def target_func(x):
    return  np.sin(5*np.pi*x) * np.log(x+2)

#
def generate_data(num_train=50): #
    num_test = 151 #
    x = np.linspace(-1, 1, num_test).reshape((-1, 1))
    y = target_func(x)
    idx = np.random.choice(num_test, num_train,  replace=False)

    #
    x_train, y_train = x[idx], y[idx]
    return x_train, y_train, x, y

#
num_train = 50
#
x_train, y_train, x_ref, y_ref = generate_data(num_train)

plt.figure()
plt.plot(x_train, y_train, 'bo',label='Sample Points')
plt.plot(x_ref, y_ref, 'k-',label='True Function')
plt.legend()
plt.show()
```



1.3

[5]:
```
## ----  ---
'''
layers=[1, 20, 20, 1],
actn='tanh',
epochs=30000
'''

#
def train_model(x_train, y_train, layers=[1, 20, 20, 1], actn='tanh',␣
 ↪epochs=30000):
    #
    model = FNN(layers, actn)
    model = model.to(device)
    # print(model.state_dict()['linear.2.weight'].shape)

    x_train, y_train = torch.tensor(x_train, dtype=torch.float32), \
        torch.tensor(y_train, dtype=torch.float32)
    x_train, y_train = x_train.to(device), y_train.to(device)
```

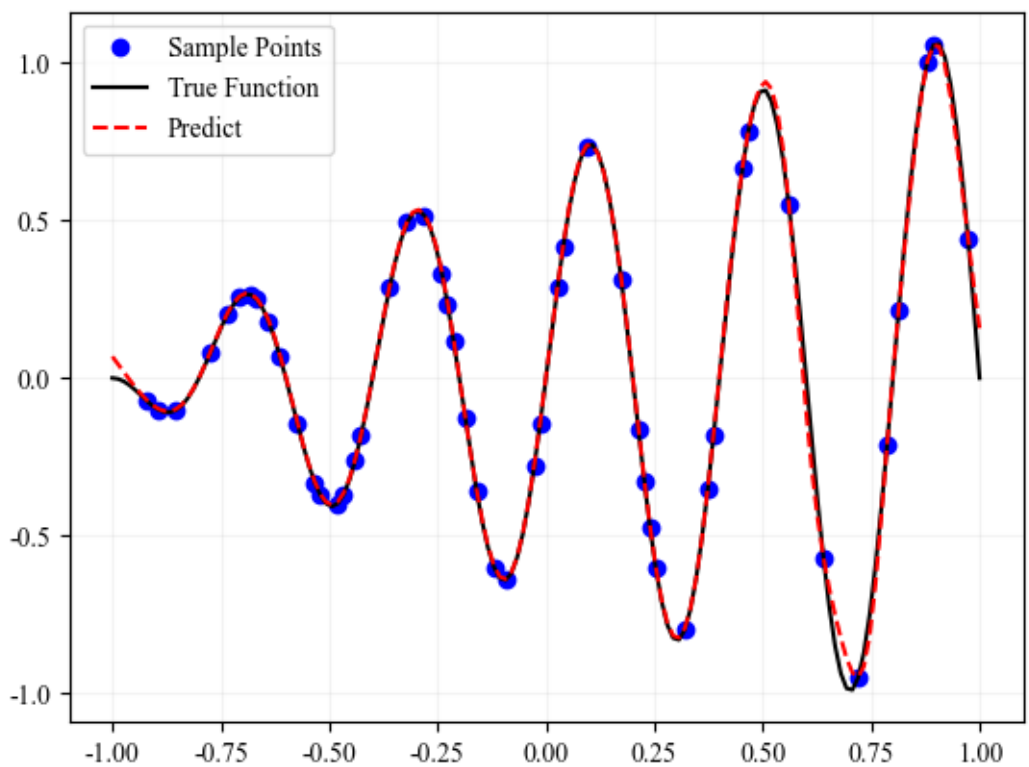
```
    opt = torch.optim.Adam(model.parameters(), lr=1.0e-3)
    loss_fn = nn.MSELoss()

    loss_history = []
    partial_loss_history = []

    start_time=time.time()

    for epoch in range(0,epochs+1):
        y_pred = model(x_train)
        loss = loss_fn(y_pred, y_train)

        opt.zero_grad() #
        loss.backward()
        opt.step()
        # loss_history.append(loss.item())

        if epoch%100 == 0:
            loss_history.append(loss.item())

        if epoch%1000 == 0:
            # print('Steps: %d, loss: %.3e'%(epoch, loss.item()))
            partial_loss_history.append(loss.item())

    end_time = time.time()

    #
    train_time = end_time - start_time
    #
    loss_history = np.array(loss_history).reshape(-1,1)
    return model.cpu(), loss_history, partial_loss_history, train_time
```

[6]:
```
default_epochs = train_model.__defaults__[2]
default_n = np.array([i for i in range(0,default_epochs+1,1000)])
default_n
```

[6]:
```
array([    0,  1000,  2000,  3000,  4000,  5000,  6000,  7000,  8000,
        9000, 10000, 11000, 12000, 13000, 14000, 15000, 16000, 17000,
       18000, 19000, 20000, 21000, 22000, 23000, 24000, 25000, 26000,
       27000, 28000, 29000, 30000])
```

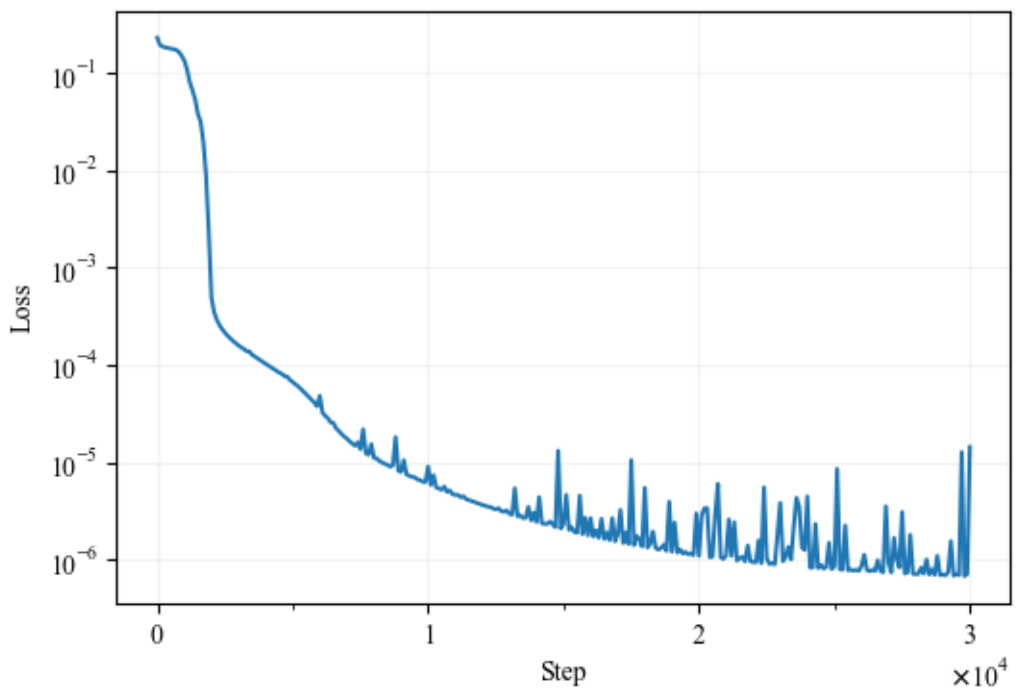
[7]:
```
## ----  ----
#
layers = [1] + [20]*2 + [1]
#
actn = 'tanh'
#
```

```
epochs = 30000


model, loss, partial_loss, train_time= train_model(x_train, y_train, layers,␣
 ↪actn, epochs)

x_ref = torch.tensor(x_ref, dtype=torch.float32)
y_test = model(x_ref)

plt.figure()

plt.plot(x_train, y_train, 'bo',label='Sample Points')
plt.plot(x_ref, y_ref, 'k-',label='True Function')

plt.plot(x_ref, y_test.detach().numpy(), 'r--',label="Predict")

plt.legend()
plt.grid(ls='-', alpha=0.15)
plt.show()
```



[8]:
```
#
partial_100n = np.array([j for j in range(0,epochs+1,100)])
```

```
fig, ax = plt.subplots(figsize=(6,4))
plt.plot(partial_100n, loss)

ax.set_yscale('log')
ax.yaxis.set_major_locator(ticker.LogLocator(base=10))
ax.yaxis.set_minor_locator(ticker.NullLocator())

ax.xaxis.set_major_locator(ticker.MultipleLocator(10000))
ax.xaxis.set_minor_locator(ticker.MultipleLocator(5000))

formatter = ticker.ScalarFormatter(useMathText=True)
formatter.set_powerlimits((4,4))
ax.xaxis.set_major_formatter(formatter)

plt.xlabel('Step')
plt.ylabel('Loss')
plt.grid(ls='-', alpha=0.15)
plt.show()
```



1.4

[9]:
```
##

def evaluate(model):

    x_test=np.linspace(-1,1,151).reshape((-1,1))
    y_true=target_func(x_test)

    x_test_tensor=torch.tensor(x_test, dtype=torch.float32)

    with torch.no_grad():
        y_pred=model.cpu()(x_test_tensor)

    mse=mean_squared_error(y_true,y_pred)
    mae=mean_absolute_error(y_true,y_pred)

    return x_test, y_true, y_pred, mse, mae

test = evaluate(model)
print(dict(zip(['MSE','MAE'],test[3:])))
```

```
{'MSE': 0.0008697051792840281, 'MAE': 0.01467911139969052}
```

[10]:
```
## ----  ----

#  -
def plot_curve(x_true,y_true,y_pred,title):
    plt.figure(figsize=(6,4))
    plt.plot(x_true,y_true,'k-',label="True Function")
    plt.plot(x_true,y_pred,'r--',label="Predict")
    plt.title(title)
    plt.grid(ls='-', alpha=0.15)
    plt.legend()
    plt.show()


#
def plot_loss(loss_dict, title):
    fig, ax = plt.subplots(figsize=(6,4))
    for key, loss in loss_dict.items():
        ax.plot(default_n, loss, label=key)

    ax.set_yscale('log')
    ax.yaxis.set_major_locator(ticker.LogLocator(base=10))
    ax.yaxis.set_minor_locator(ticker.NullLocator())

    ax.xaxis.set_major_locator(ticker.MultipleLocator(10000))
```

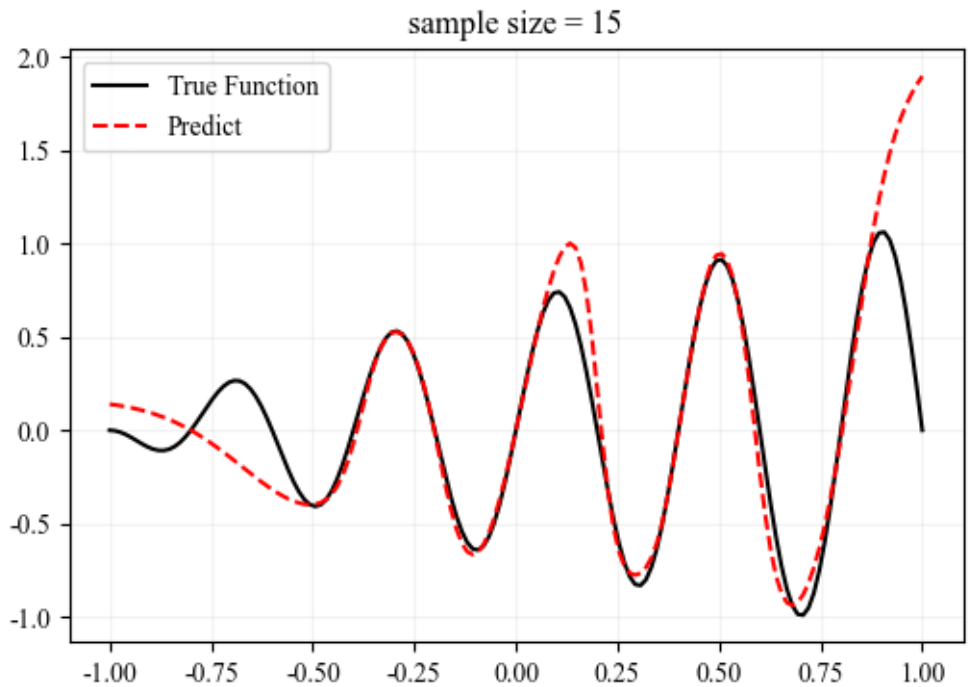
```
    ax.xaxis.set_minor_locator(ticker.MultipleLocator(5000))

    formatter = ticker.ScalarFormatter(useMathText=True)
    formatter.set_powerlimits((4,4))
    ax.xaxis.set_major_formatter(formatter)

    plt.xlabel('Step')
    plt.ylabel('Loss')
    plt.grid(ls='-', alpha=0.15)
    plt.title(title)
    plt.legend()
    plt.show()

#
def plot_loss_100(loss, key):
    fig, ax = plt.subplots(figsize=(6,4))
    ax.plot(partial_100n, loss, label=key)

    ax.set_yscale('log')
    ax.yaxis.set_major_locator(ticker.LogLocator(base=10))
    ax.yaxis.set_minor_locator(ticker.NullLocator())

    ax.xaxis.set_major_locator(ticker.MultipleLocator(10000))
    ax.xaxis.set_minor_locator(ticker.MultipleLocator(5000))

    formatter = ticker.ScalarFormatter(useMathText=True)
    formatter.set_powerlimits((4,4))
    ax.xaxis.set_major_formatter(formatter)

    plt.xlabel('Step')
    plt.ylabel('Loss')
    plt.grid(ls='-', alpha=0.15)
    # plt.title(title)
    plt.legend()
    plt.show()


#
def plot_train_time(dict, time_list, xlabel):
    pro = [str(s) for s in dict]

    plt.figure(figsize=(6,4))

    colors = plt.cm.Blues(np.linspace(0.4,0.9,len(pro)))
    bars = plt.bar(pro,time_list, width=0.6, color=colors,
                   edgecolor='black', linewidth=1)
```

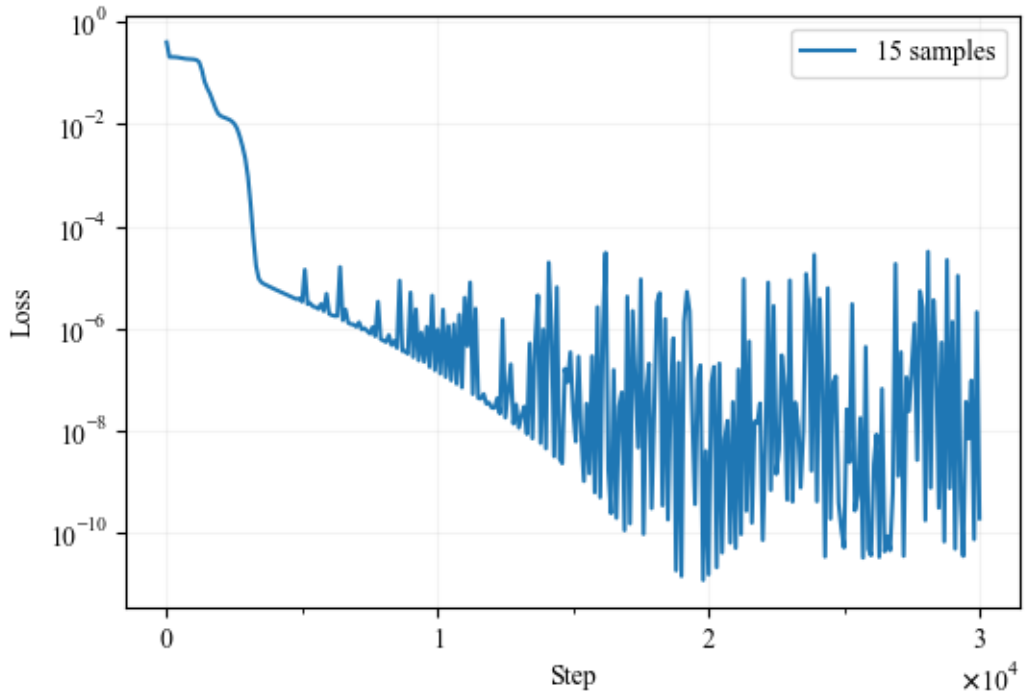
```
    plt.xlabel(xlabel)
    plt.ylabel("Training Time (s)")
    plt.ylim(bottom=min(time_list)-5)

    for bar in bars:
        height = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2,
                 height + 0.02, f"{height:.3f}", ha='center', va='bottom')

    plt.grid(axis='y', linestyle='--', alpha=0.15)
    plt.tight_layout()
    plt.show()
```

1.5 1

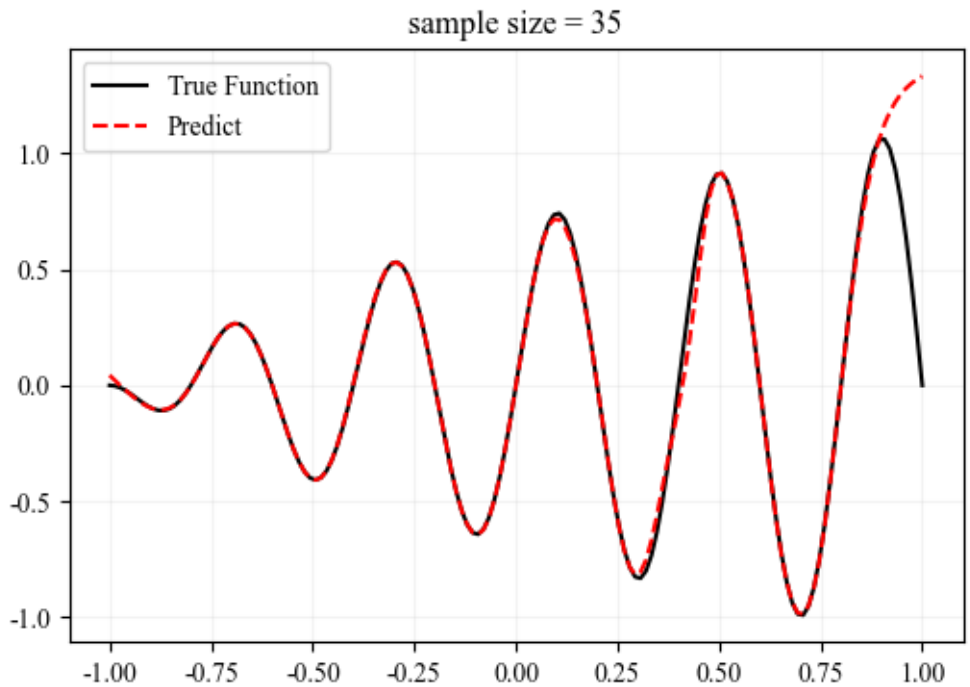
[11]:
```
## ----   ----
#
sample_sizes = [15, 35, 55, 75, 100]

#   -MSE-MAE
results_q1 = []
#   -
loss_dict_q1 = {}

for n in sample_sizes:
    print('    ',n)
    #
    x_train_q1, y_train_q1, x_ref_q1, y_ref_q1 = generate_data(n)
    model_q1, loss_q1, partial_loss_q1, train_time_q1= train_model(x_train_q1,␣
 ↪y_train_q1)

    #
    x_test_q1, y_true_q1, y_pred_q1, mse_q1, mae_q1 = evaluate(model_q1)
    results_q1.append([n, mse_q1, mae_q1])

    loss_dict_q1[str(n)+' samples'] = partial_loss_q1

    plot_curve(x_test_q1, y_true_q1, y_pred_q1,f"sample size = {n}")
    plot_loss_100(loss_q1, str(n)+' samples')
```

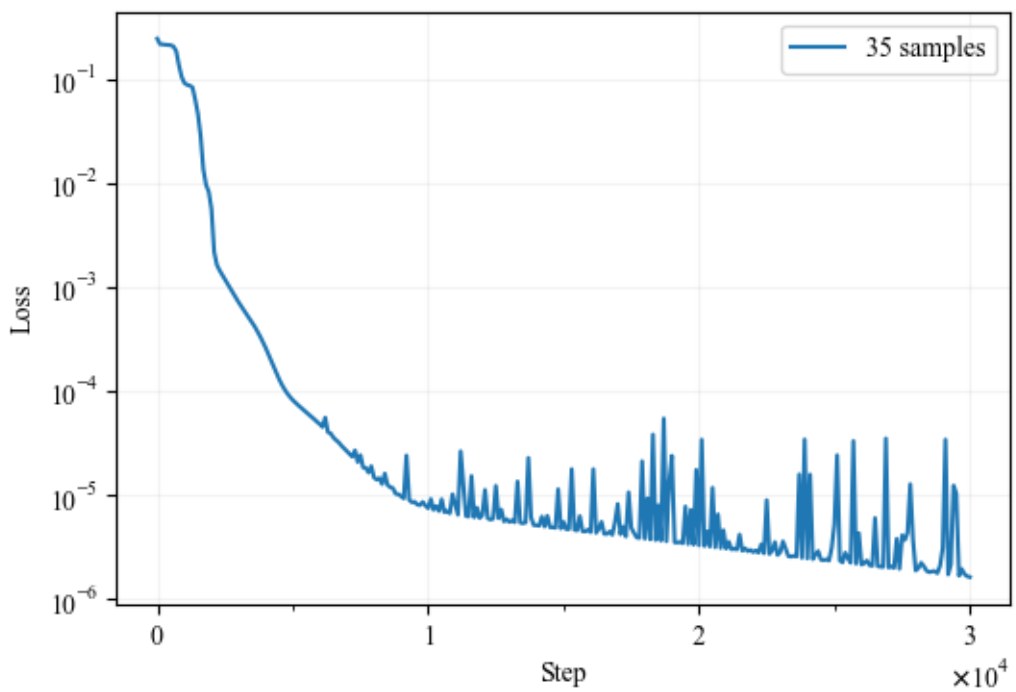
```
    15
```


sample size = 15
True Function
Predict
2.0
1.5
1.0
0.5
0.0
-0.5
-1.0
-1.00
-0.75
-0.50
-0.25
0.00
0.25
0.50
0.75
1.00

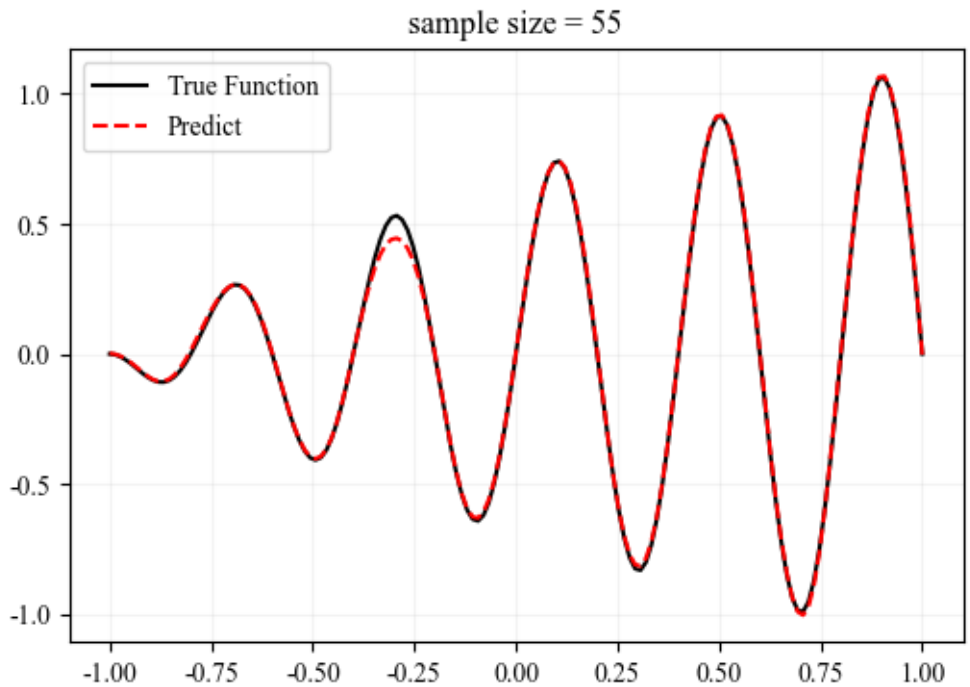

15 samples
Loss
Step
10^{0}
10^{-2}
10^{-4}
10^{-6}
10^{-8}
10^{-10}
0
1
2
3
$\times 10^{4}$

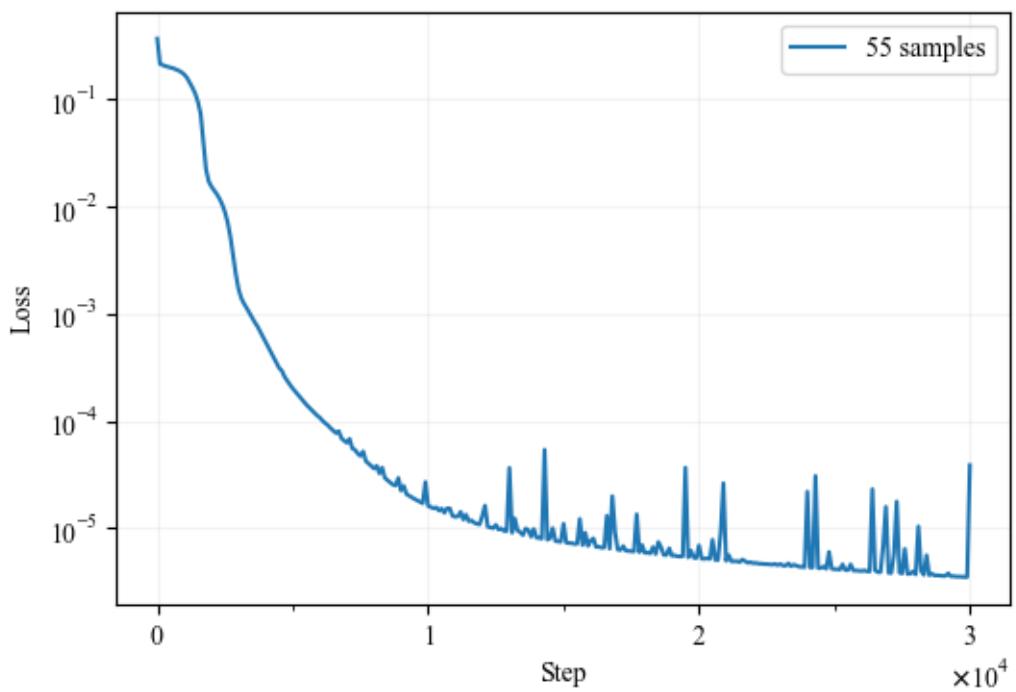
35


sample size = 35
True Function
Predict
1.0
0.5
0.0
-0.5
-1.0
-1.00
-0.75
-0.50
-0.25
0.00
0.25
0.50
0.75
1.00

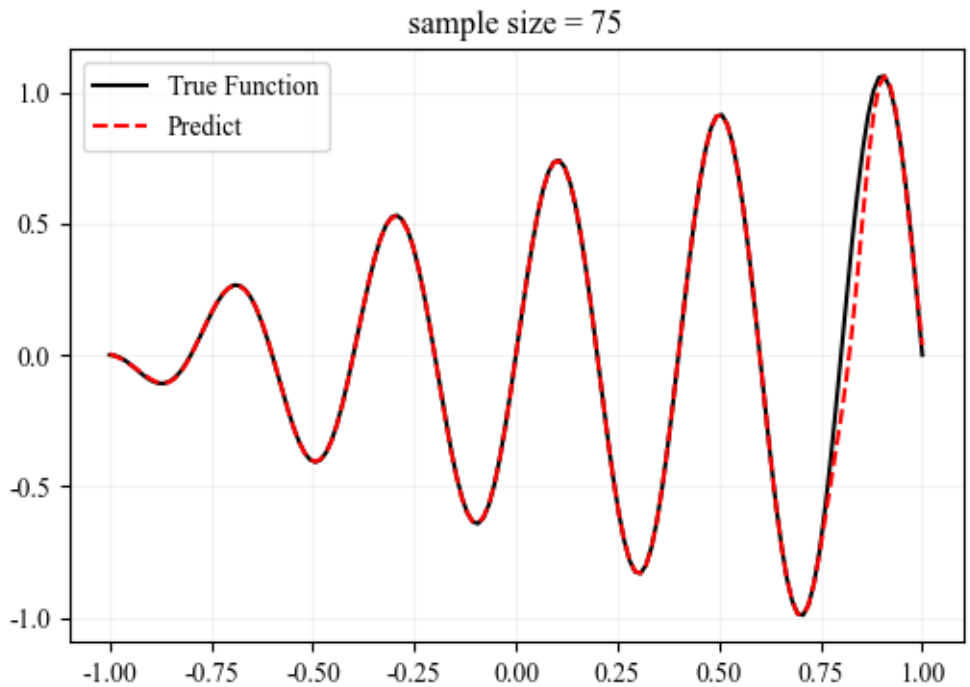

35 samples
Loss
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{-5}
10^{-6}
0
1
2
3
Step
$\times 10^4$

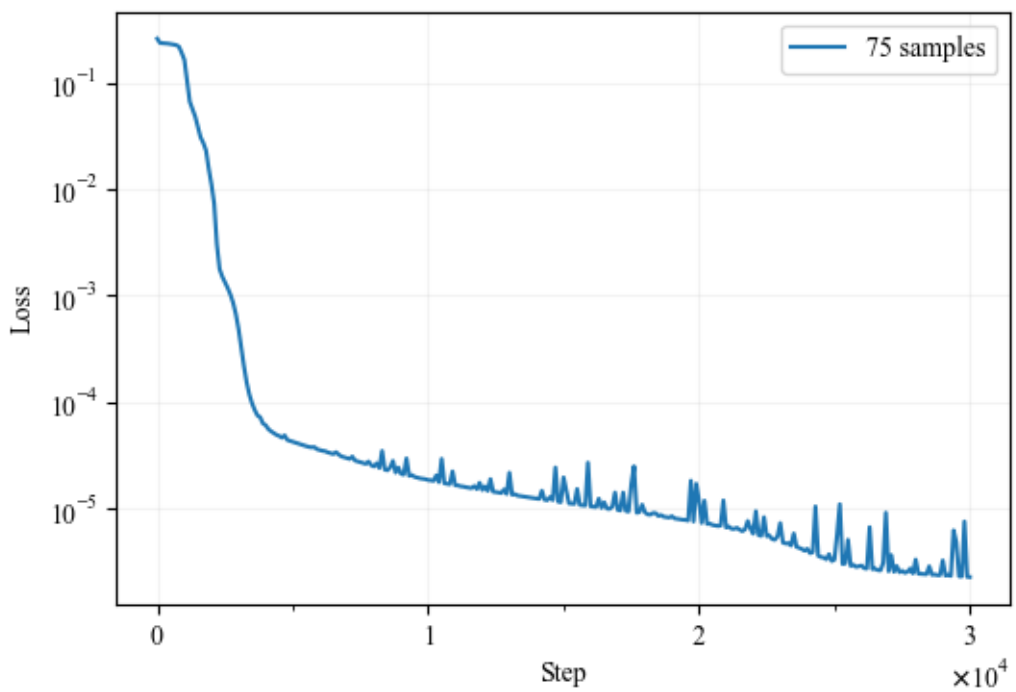
55


sample size = 55
True Function
Predict
1.0
0.5
0.0
-0.5
-1.0
-1.00
-0.75
-0.50
-0.25
0.00
0.25
0.50
0.75
1.00

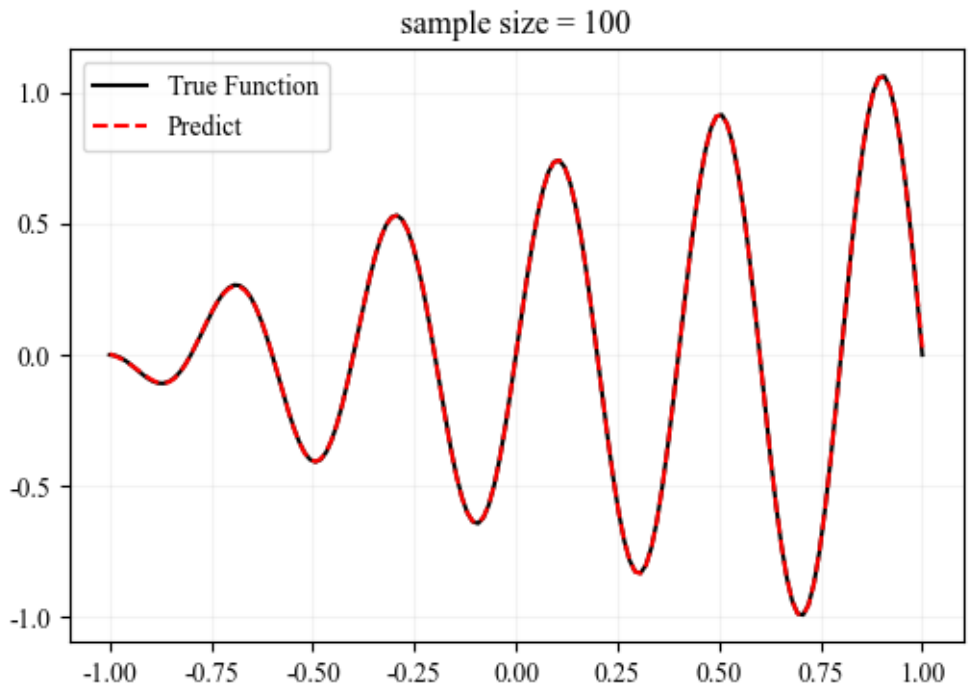

55 samples
Loss
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{-5}
0
1
2
3
Step
$\times 10^4$

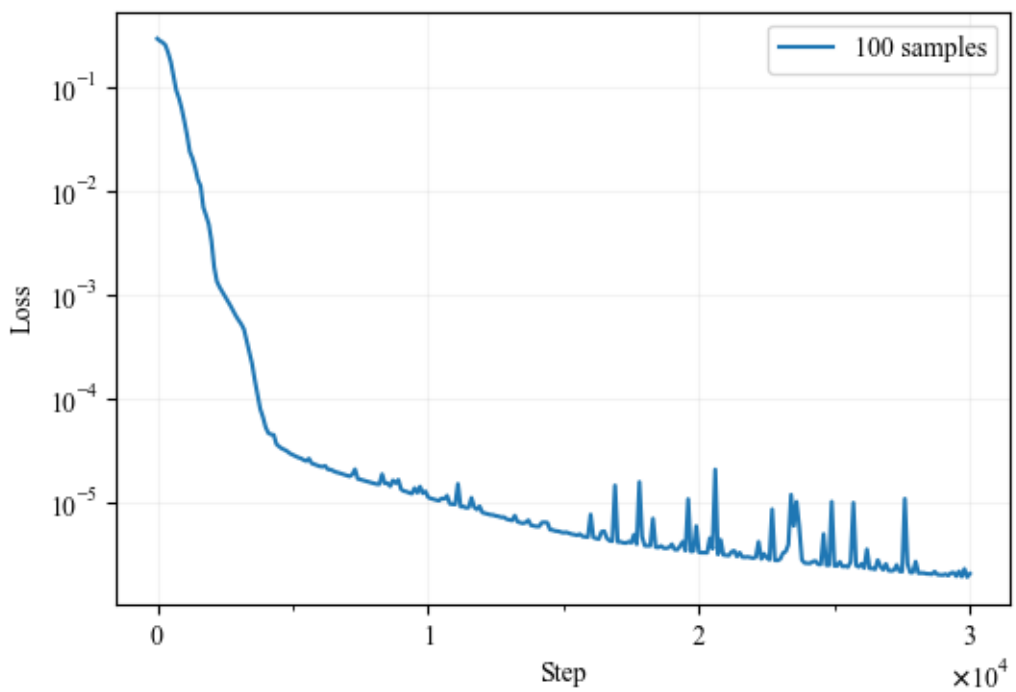
75


sample size = 75
True Function
Predict
1.0
0.5
0.0
-0.5
-1.0
-1.00
-0.75
-0.50
-0.25
0.00
0.25
0.50
0.75
1.00


75 samples
Loss
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{-5}
0
1
2
3
Step
$\times 10^4$

100


sample size = 100
True Function
Predict
1.0
0.5
0.0
-0.5
-1.0
-1.00
-0.75
-0.50
-0.25
0.00
0.25
0.50
0.75
1.00


100 samples
Loss
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{-5}
0
1
2
3
Step
$\times 10^{4}$

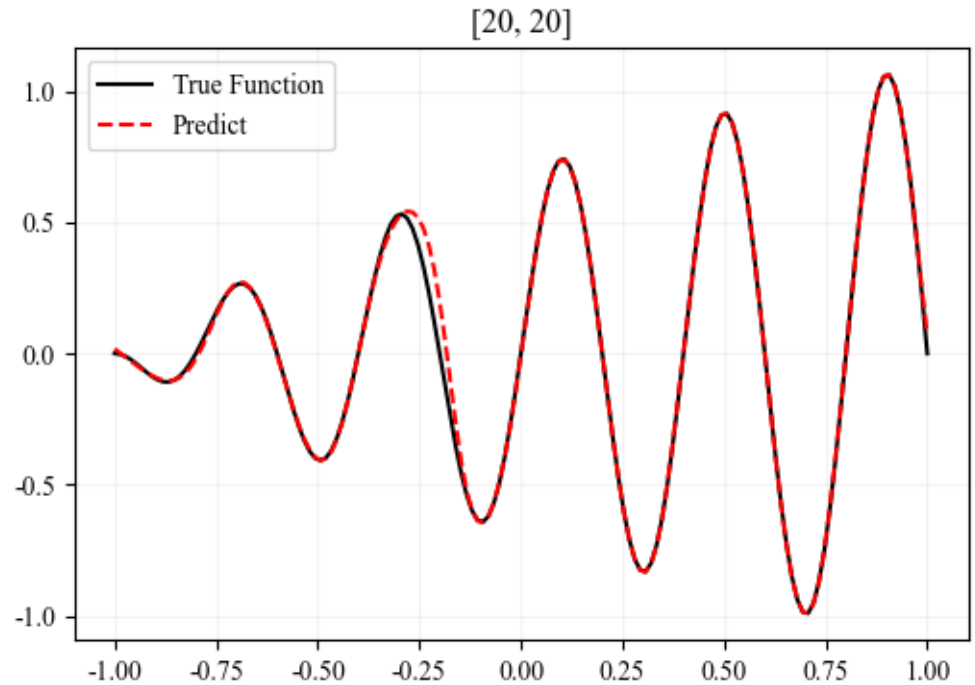
[12]:
```
#  MSE/MAE/
df_q1=pd.DataFrame(results_q1,columns=['Sample_Sizes','MSE','MAE'])
print(df_q1)


# plot_loss(loss_dict_q1, title='Loss Comparison (Different Sample Sizes)')
```

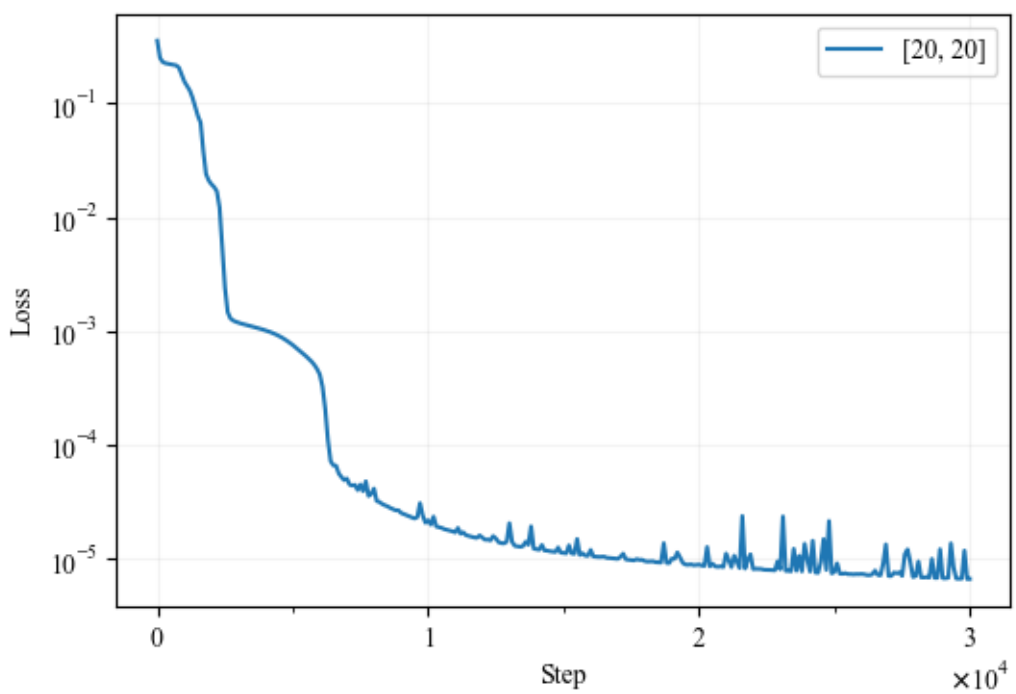
```
   Sample_Sizes       MSE       MAE
0            15  0.096760  0.155777
1            35  0.030227  0.043916
2            55  0.000338  0.010676
3            75  0.003336  0.015444
4           100  0.000008  0.001479
```

1.6 2

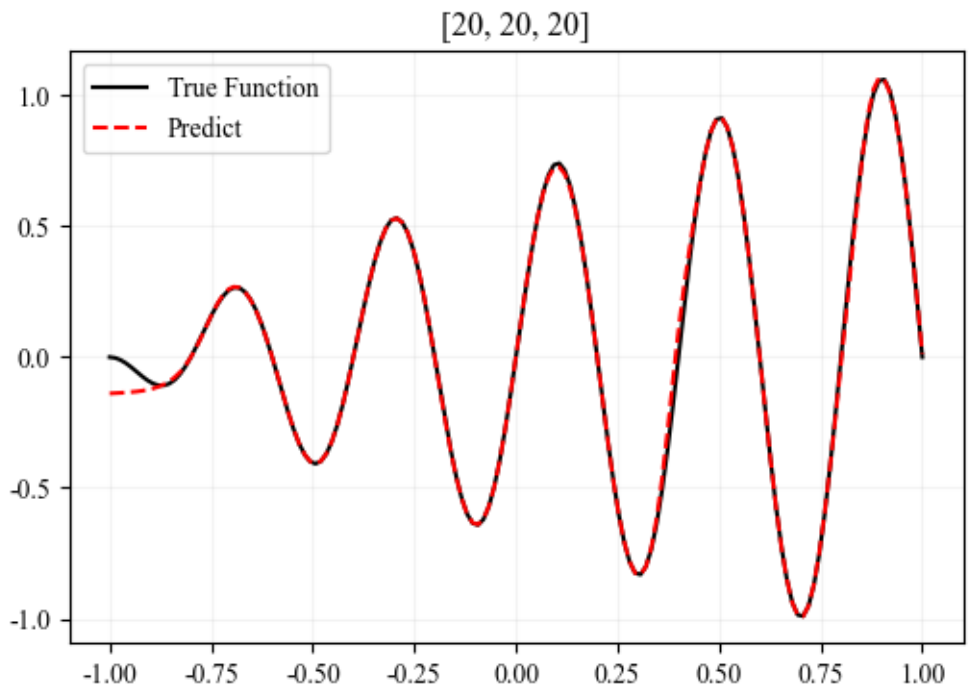
[13]:
```
## ----   ----
#
hide_layers = [[20,20], [20,20,20], [50,50], [50,50,50],
               [50,50,50,50],[100,100]]

#   -MSE-MAE-
results_q2 = []
#   -
loss_dict_q2 = {}
#   -
time_list_q2 = []

for s in hide_layers:
    layers_q2 = [1] + s +[1]
    print('    ',layers_q2)
    #
    x_train_q2, y_train_q2, x_ref_q2, y_ref_q2 = generate_data()
    model_q2, loss_q2, partial_loss_q2, train_time_q2 = train_model(x_train_q2,␣
 ↪y_train_q2, layers=layers_q2)

    #
    x_test_q2, y_true_q2, y_pred_q2, mse_q2, mae_q2 = evaluate(model_q2)

    print('MSE:',mse_q2,'MAE:',mae_q2,'Train_time:',train_time_q2)

    results_q2.append([s, mse_q2, mae_q2, train_time_q2])
    loss_dict_q2[str(s)] = partial_loss_q2
    time_list_q2.append(train_time_q2)

    plot_curve(x_test_q2, y_true_q2, y_pred_q2, str(s))
    plot_loss_100(loss_q2, str(s))
```

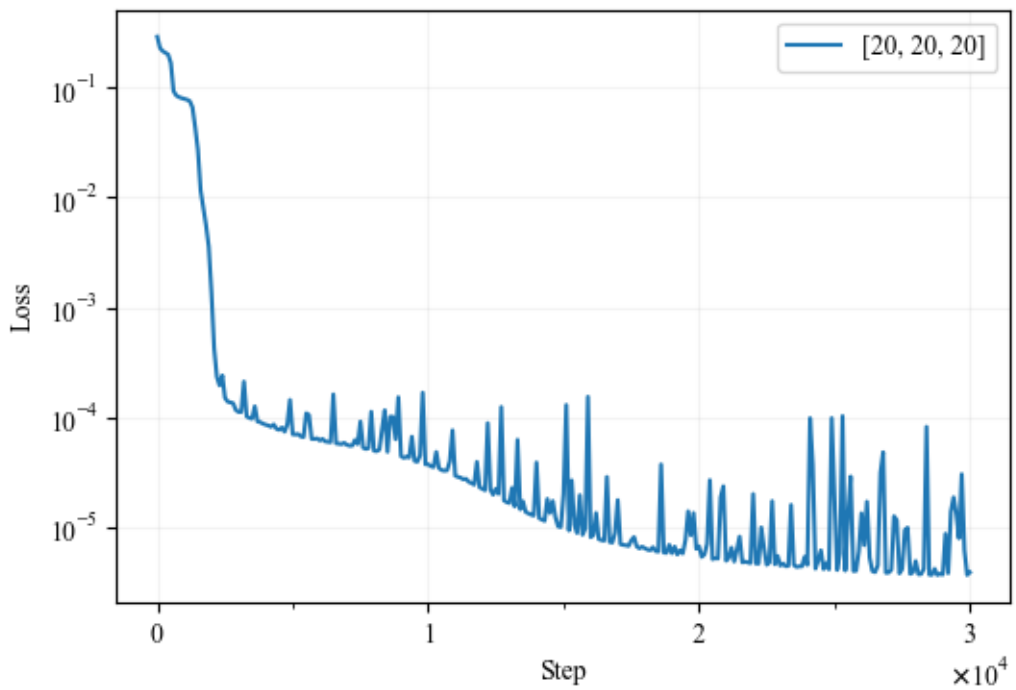
```
    [1, 20, 20, 1]
MSE: 0.0014344984092600967 MAE: 0.013139642609343402 Train_time:
33.007731914520264
```



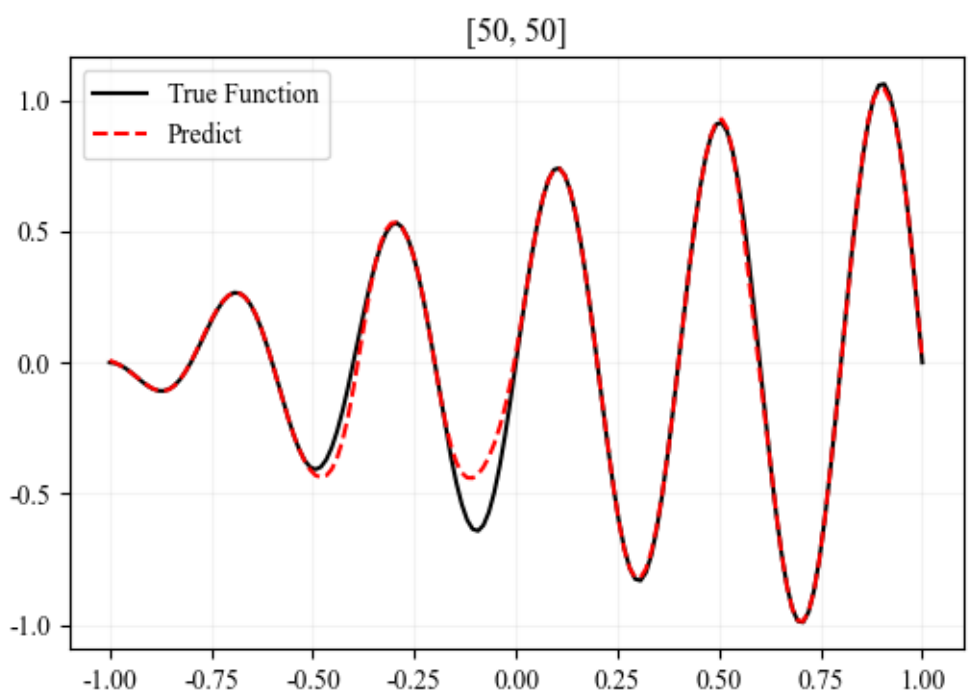


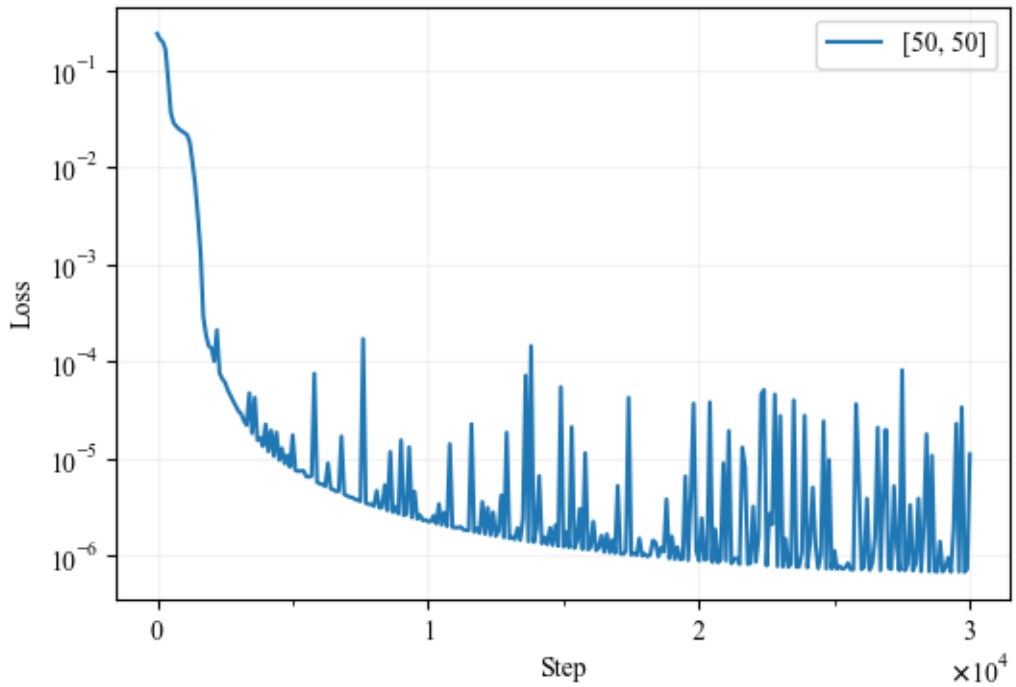
```
    [1, 20, 20, 20, 1]
MSE: 0.0008538160273630477 MAE: 0.011540786821722243 Train_time:
37.89703369140625
```



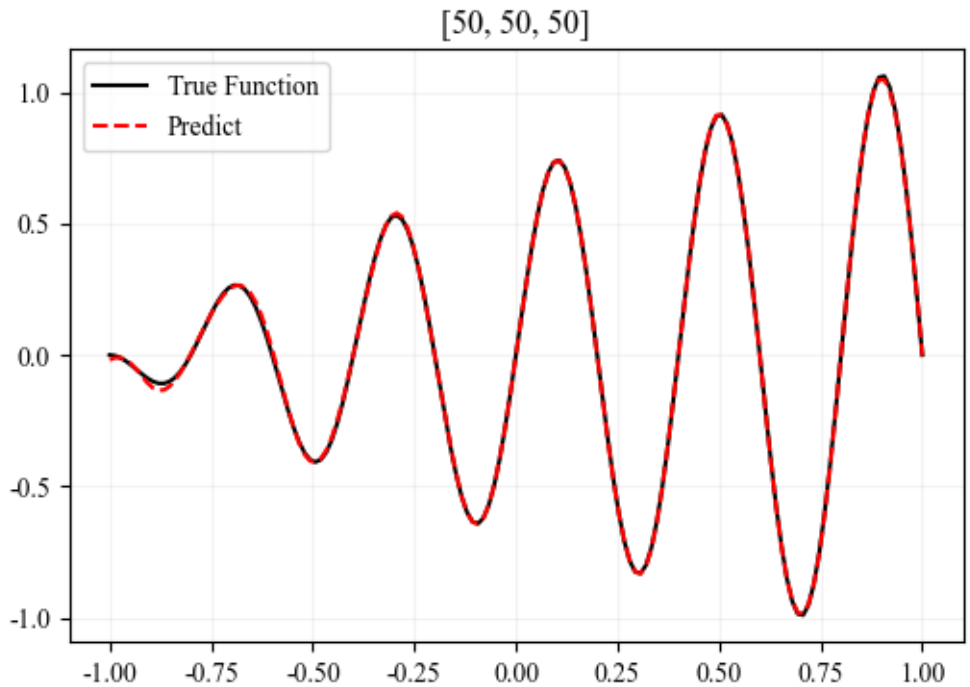


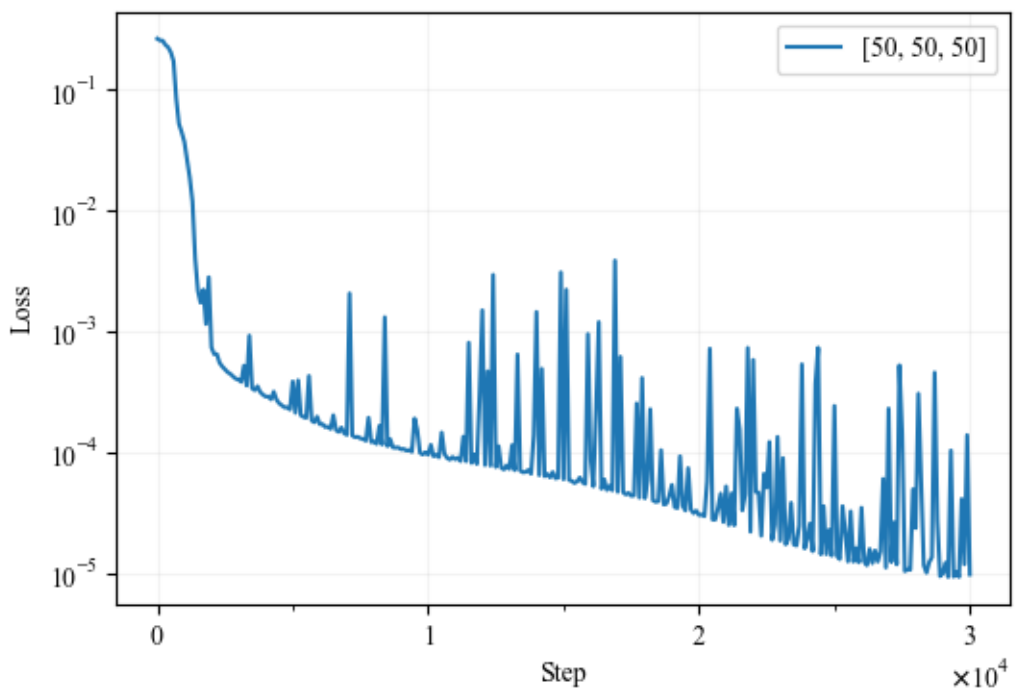

```
    [1, 50, 50, 1]
MSE: 0.002532544748000685 MAE: 0.0212552503731735 Train_time: 31.040125846862793
```





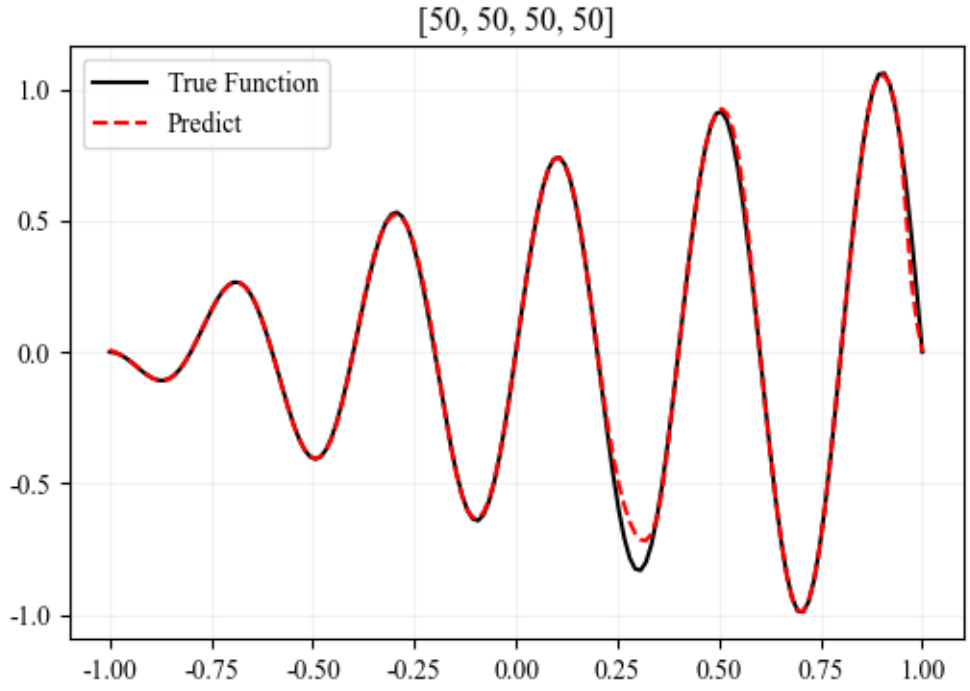

```
    [1, 50, 50, 50, 1]
MSE: 5.766674981161655e-05 MAE: 0.005104548889685566 Train_time:
37.29765272140503
```

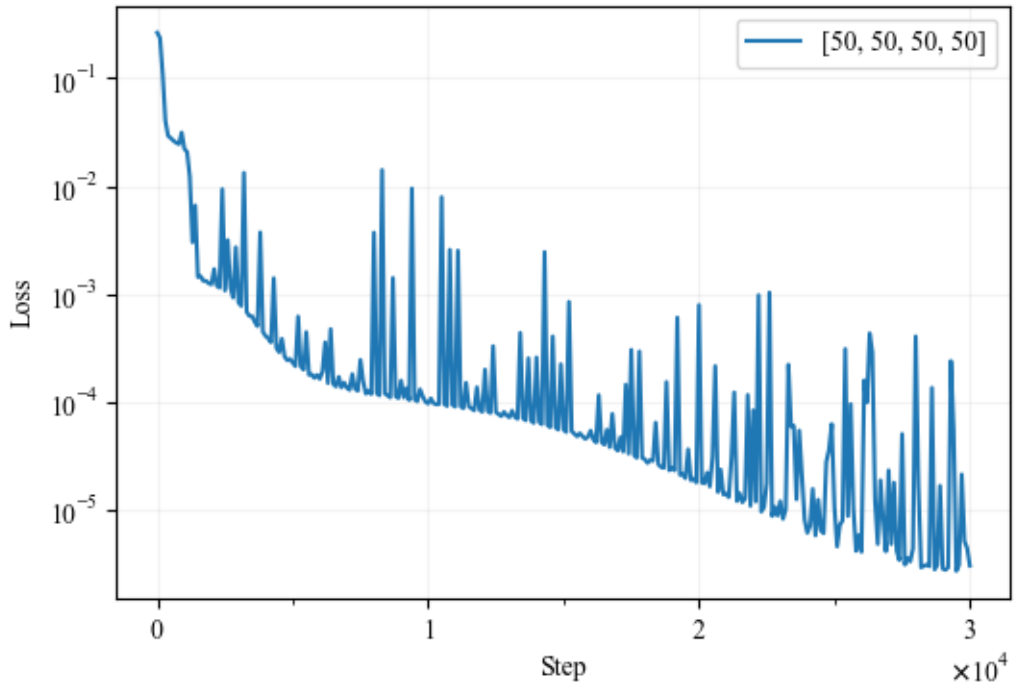




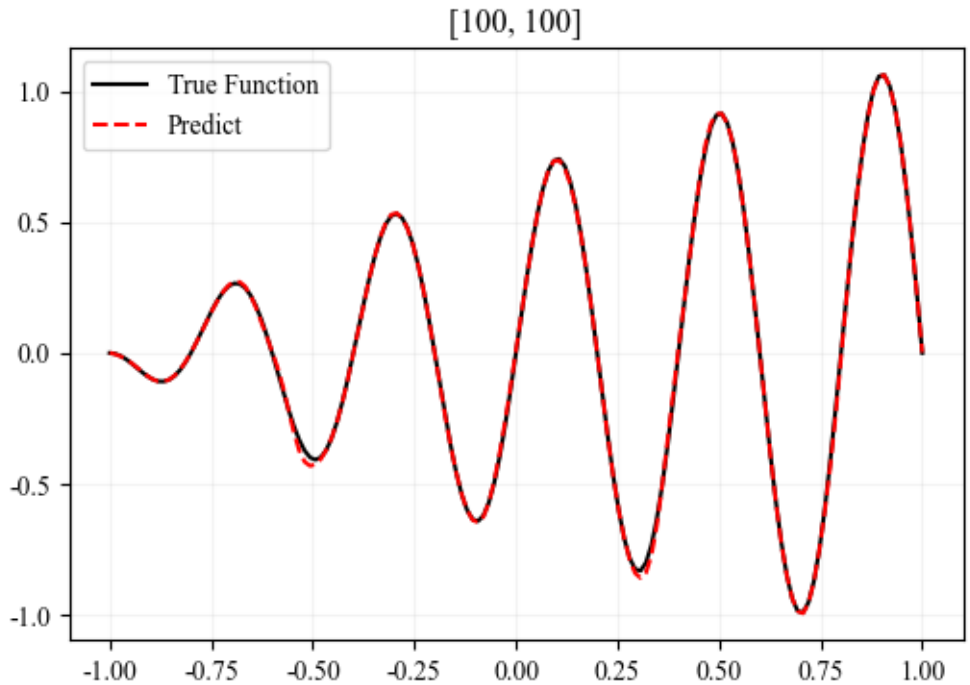

```
    [1, 50, 50, 50, 50, 1]
MSE: 0.0010754941354062168 MAE: 0.013012944988682426 Train_time:
```

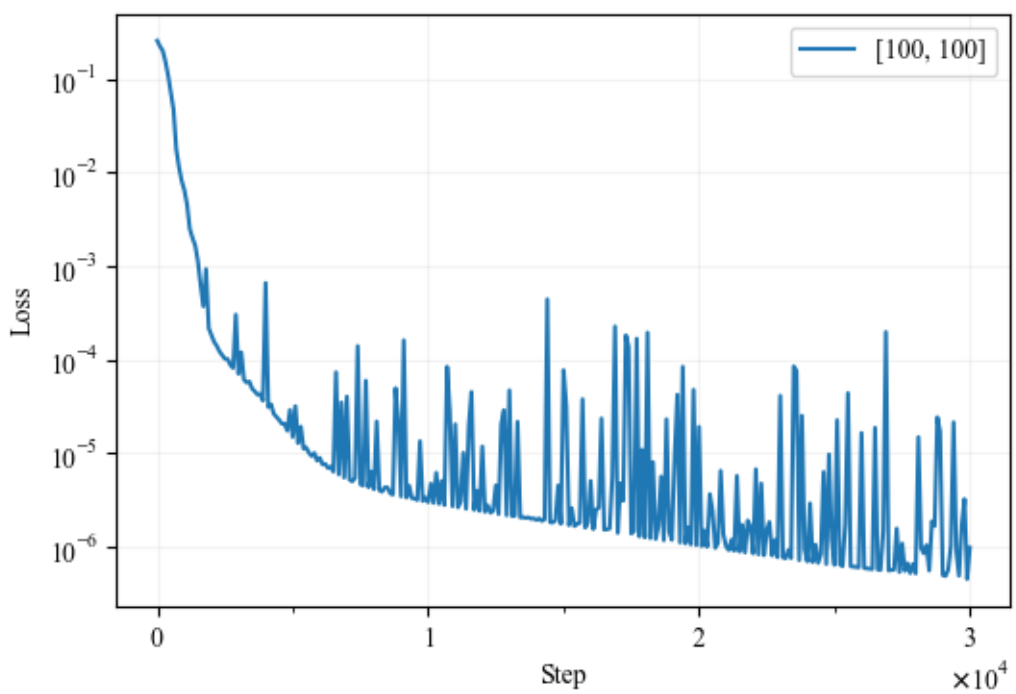
45.31252408027649





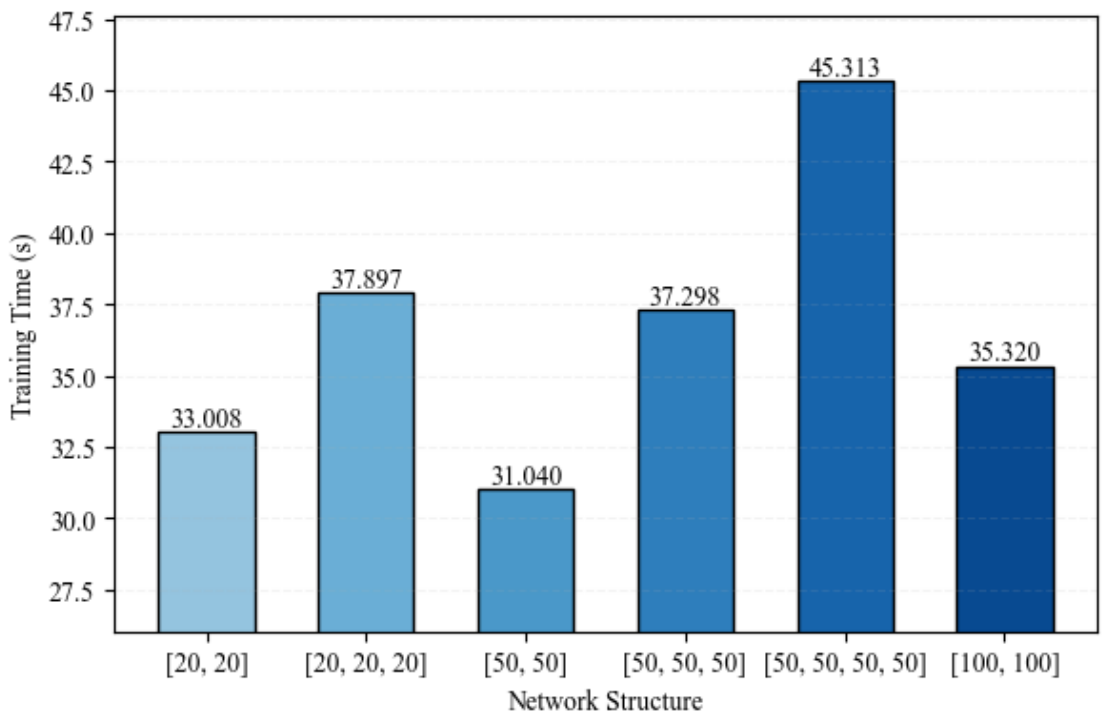
```
    [1, 100, 100, 1]
MSE: 0.00010211313067882455 MAE: 0.005346811880509773 Train_time:
35.319626569747925
```





[14]:
```
#  MSE/MAE/  /
df_q2=pd.DataFrame(results_q2,columns=['Hide_Layers','MSE','MAE','Train_time'])
print(df_q2)


# plot_loss(loss_dict_q2, title='Loss Comparison (Different Network␣
↪Structures)')

plot_train_time(hide_layers, time_list_q2, xlabel="Network Structure")
```

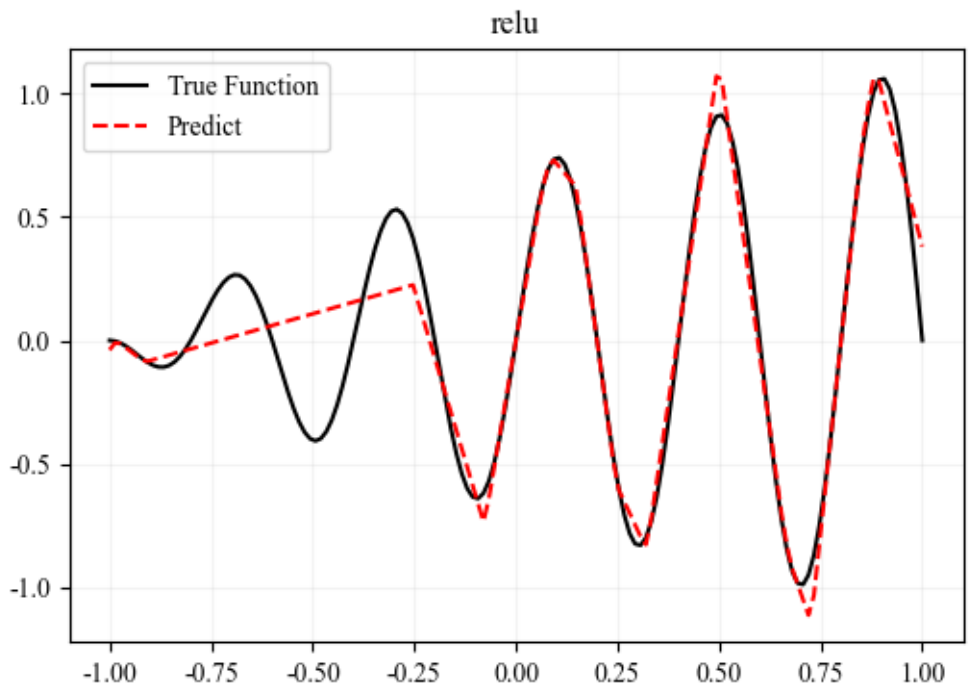
```
        Hide_Layers       MSE       MAE  Train_time
0          [20, 20]  0.001434  0.013140   33.007732
1      [20, 20, 20]  0.000854  0.011541   37.897034
2          [50, 50]  0.002533  0.021255   31.040126
3      [50, 50, 50]  0.000058  0.005105   37.297653
4  [50, 50, 50, 50]  0.001075  0.013013   45.312524
5        [100, 100]  0.000102  0.005347   35.319627
```

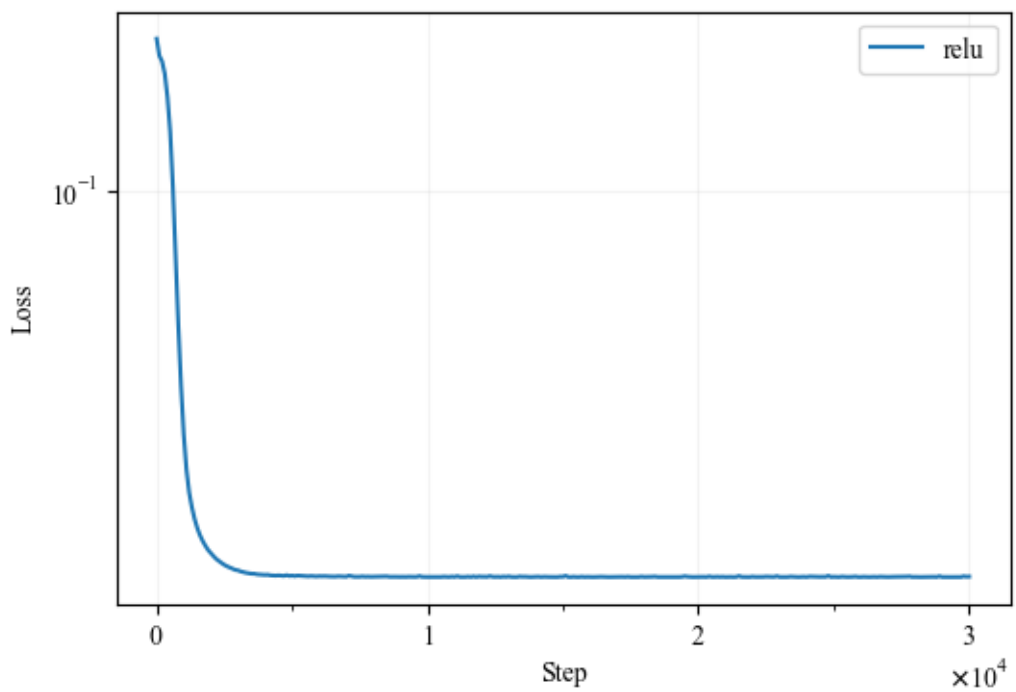


1.7 3

[15]:
```
## ----   ----
#
activations = ['relu', 'tanh', 'sigmoid']

#   -MSE-MAE
results_q3 = []
#   -
loss_dict_q3 = {}
#   -
time_list_q3 = []

for act in activations:
    print('    ',act)
    #
    x_train_q3, y_train_q3, x_ref_q3, y_ref_q3 = generate_data()
    model_q3, loss_q3, partial_loss_q3, train_time_q3 = train_model(x_train_q3,␣
 ↪y_train_q3, actn = act)

    #
    x_test_q3, y_true_q3, y_pred_q3, mse_q3, mae_q3 = evaluate(model_q3)

    print('Structure:', act, "MSE:", mse_q3, "MAE:", mae_q3, 'Train_Time:',␣
 ↪train_time_q3)

    results_q3.append([act, mse_q3, mae_q3])
    loss_dict_q3[act] = partial_loss_q3
    time_list_q3.append(train_time_q3)

    plot_curve(x_test_q3, y_true_q3, y_pred_q3, act)
    plot_loss_100(loss_q3, act)
```

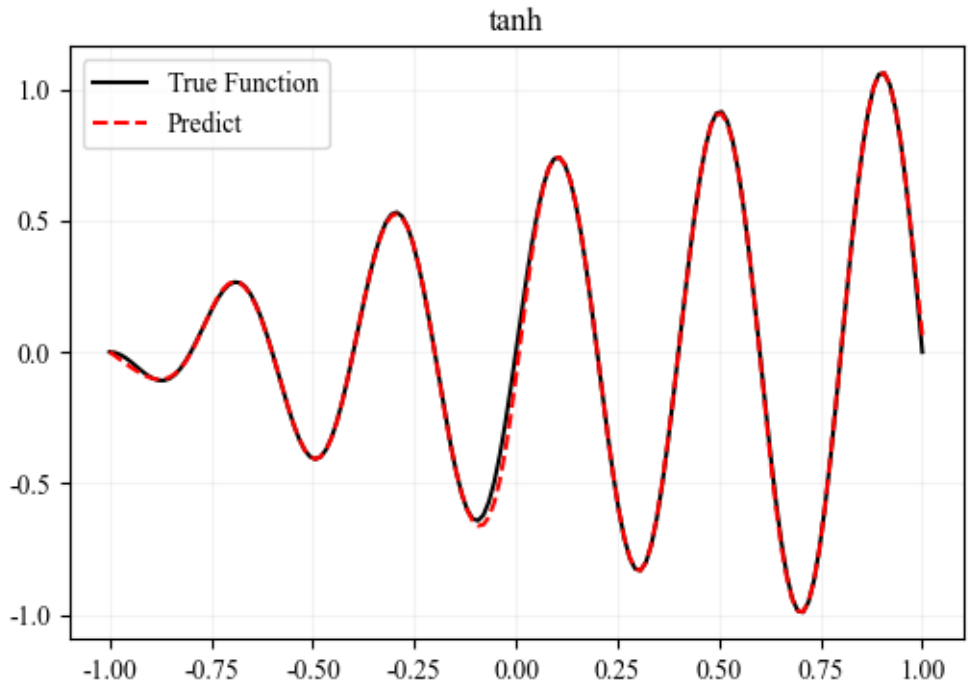
```
    relu
Structure: relu MSE: 0.02646640944651834 MAE: 0.10475927160310698 Train_Time:
33.64403200149536
```

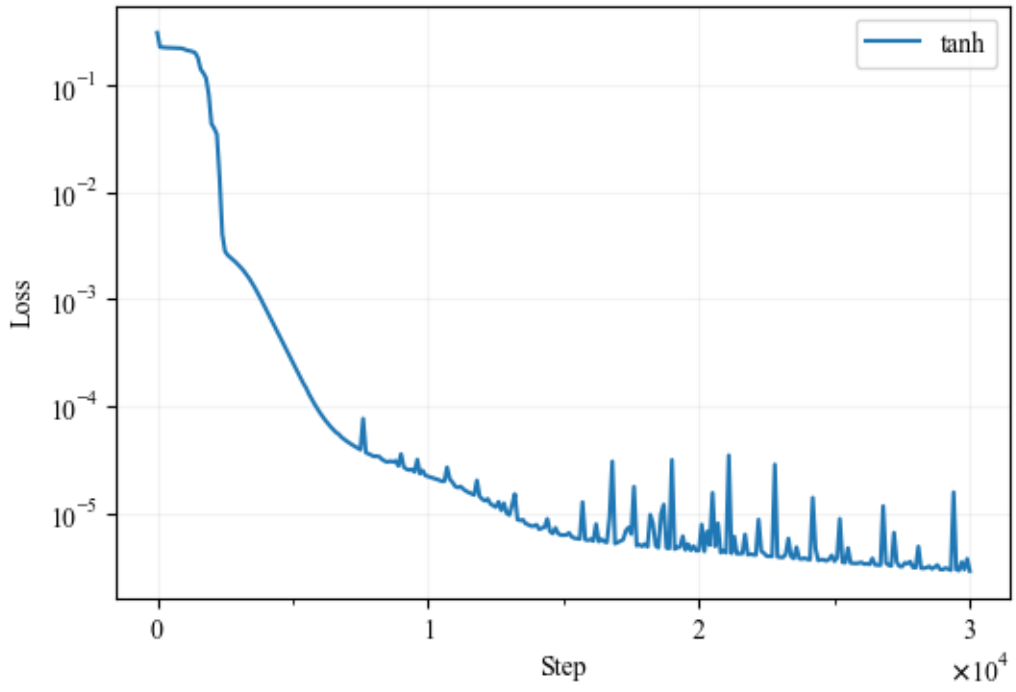




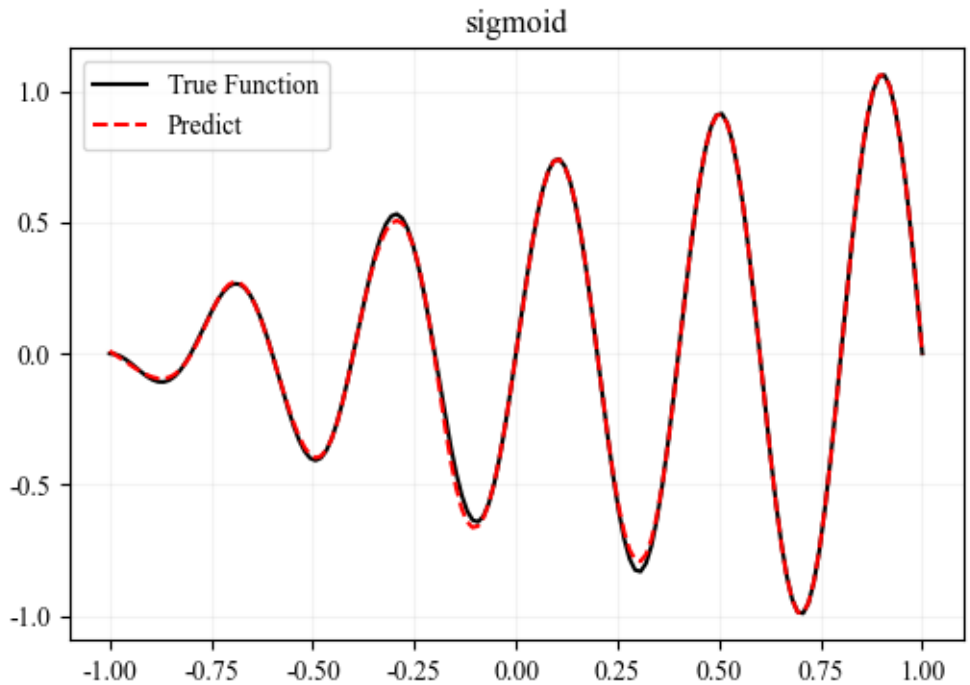

```
    tanh
Structure: tanh MSE: 0.00047165966809540753 MAE: 0.008771070739469202
```

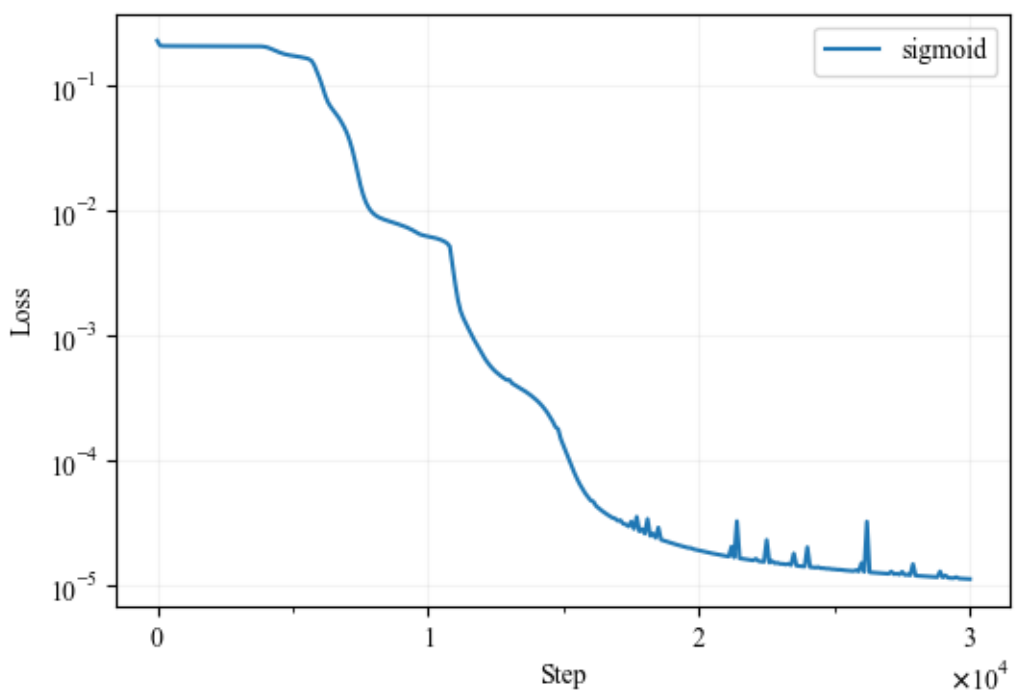
```
Train_Time: 31.772510290145874
```

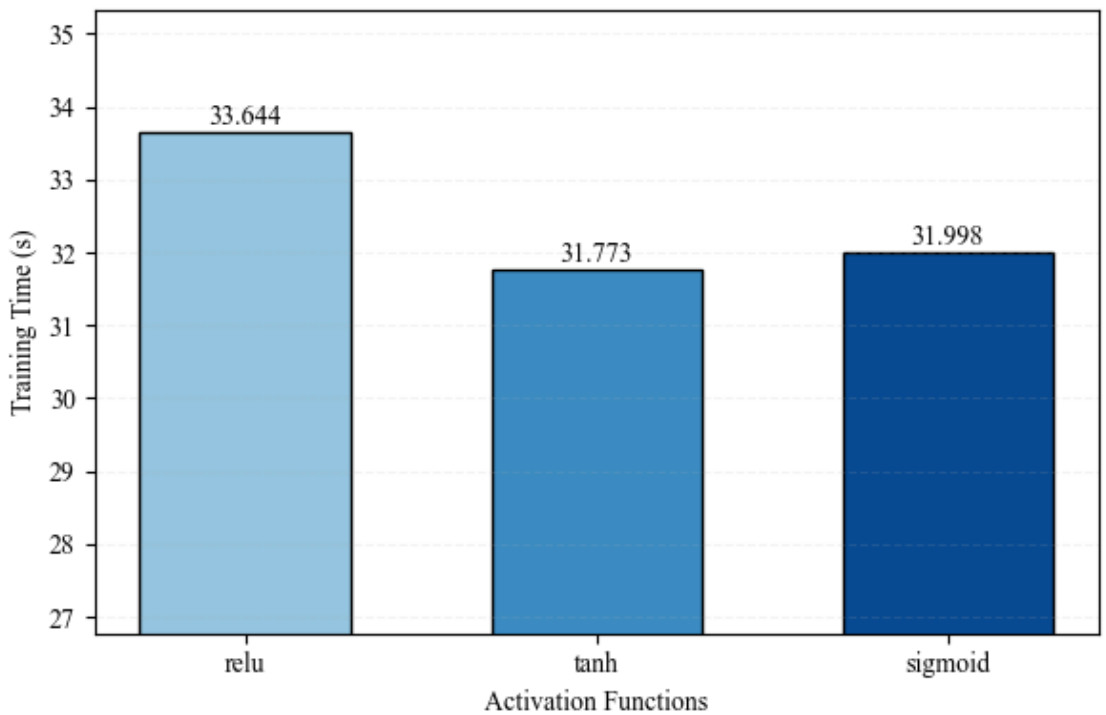




```
    sigmoid
Structure: sigmoid MSE: 0.00018068312836717345 MAE: 0.007514278281851676
Train_Time: 31.997560739517212
```





[16]:
```
#  MSE/MAE/  /
df_q3=pd.DataFrame(results_q3,columns=['Activation','MSE','MAE'])
print(df_q3)

# plot_loss(loss_dict_q3, title='Loss Comparison (Different Activation␣
↪Functions)')

plot_train_time(activations, time_list_q3,xlabel="Activation Functions")
```

```
  Activation       MSE       MAE
0       relu  0.026466  0.104759
1       tanh  0.000472  0.008771
2    sigmoid  0.000181  0.007514
```